遗传算法计算最短工期

1. 2025.11.25
   a. 优化支管起点和路径查找逻辑；
   b. 完善遗传算法，实现网格动态占用释放机制：设计双层数据结构，实现焊口级别的网格动态释放。
   c. 优化网格占用算法
      i. 跨工区段分配：引入银行家算法的资源预判机制，优先满足资源充裕的任务。
      ii. 段内网格切换：设计了简化版银行家算法结合短时等待队列，解决连续施工中的资源衔接。
      iii. 工区内调度：结合负载均衡与启发式规则，优化初始任务分配，减少局部拥堵。

2. 2025.11.14
   a. 增加工区密度约束，将工区划分为多个网格，每个网格允许固定的焊工工作；
   b. 为了区分管线内主管和支管，增加拓扑构建逻辑，将管线划分为主管段和支管段，用于构建染色体。

遗传算法.zip
112.4MB

3. 2025.11.10
   a. 增加试压包和单元约束，确保焊工在完成试压包内管线后优先焊接同一单元的其他管线。
   b. 将选择算子由轮盘赌选择改为锦标赛选择。

复制代码

```
"""
染色体解码模块（带试压包硬约束）
职责：将染色体（管线排列）转换为实际调度方案，强制试压包优先规则
"""
from typing import List, Set, Dict
from collections import defaultdict
from ..models import Schedule, Task, WorkerGroup, ProjectData, Worker
from ..config import WORKER_CONFIG


def decode_chromosome_with_package_priority(
    chromosome: List[str],
```

```
    project_data: ProjectData,
    worker_group: WorkerGroup,
    daily_capacity: float = None,
    working_hours_per_day: float = None
) -> Schedule:
    """
    解码染色体为调度方案（带试压包硬约束）

    硬约束规则：
    1. 焊工完成一条管线后，优先焊接同一试压包内的其他管线
    2. 只有当该试压包内没有可焊接管线时，才考虑其他试压包
    3. 没有试压包的管线（Unknown）放到最后处理

    参数：
        chromosome: 染色体（管线号的排列）
        project_data: 项目数据对象（ProjectData）
        worker_group: 焊工组对象
        daily_capacity: 每组每天焊接寸数
        working_hours_per_day: 每天工作小时数

    返回：
        Schedule对象，包含所有任务
    """
    # 从配置获取默认值
    from ..config import WORKING_HOURS_PER_DAY as DEFAULT_WORKING_HOURS

    if daily_capacity is None:
        # 从worker_group获取daily_capacity
        daily_capacity = getattr(worker_group, 'daily_capacity', 25)

    if working_hours_per_day is None:
        working_hours_per_day = DEFAULT_WORKING_HOURS

    # 重置所有焊工的可用时间
    worker_group.reset()

    # 创建调度方案
    schedule = Schedule()

    # 注意：染色体中的管线已经在种群初始化前预先筛选过，
    # 只包含该组能焊的管线，无需再次过滤

    # 已分配的管线集合
    assigned_pipelines: Set[str] = set()
```

```
    # 按试压包分组管线（排除Unknown）
    package_pipelines: Dict[str, List[str]] = defaultdict(list)
    unknown_pipelines: List[str] = []

    for pipeline_no in chromosome:
        pipeline = project_data.get_pipeline(pipeline_no)
        if pipeline:
            if pipeline.package_no == 'Unknown' or pipeline.package_no
is None:
                unknown_pipelines.append(pipeline_no)
            else:

package_pipelines[pipeline.package_no].append(pipeline_no)

    # 记录每个焊工当前正在处理的试压包和单元
    worker_current_package: Dict[int, str] = {}
    worker_current_unit: Dict[int, str] = {}

    # 标记哪些焊工已经没有可做的管线了
    workers_with_no_task: Set[int] = set()

    # 主循环：直到所有管线都被分配
    while len(assigned_pipelines) < len(chromosome):
        # 如果所有焊工都没有可做的管线，退出循环
        if len(workers_with_no_task) >= len(worker_group.workers):
            break

        # 找到最早可用且还有可做任务的焊工
        available_workers = [w for w in worker_group.workers if
w.worker_id not in workers_with_no_task]
        if not available_workers:
            break

        earliest_worker = min(available_workers, key=lambda w:
w.available_time)

        # 获取该焊工当前的试压包和单元
        current_package =
worker_current_package.get(earliest_worker.worker_id)
        current_unit =
worker_current_unit.get(earliest_worker.worker_id)

        # 选择该焊工能做的下一条管线（考虑队伍-单元限制）
        next_pipeline_no = _select_next_pipeline(
            current_package=current_package,
```

```
            current_unit=current_unit,
            package_pipelines=package_pipelines,
            unknown_pipelines=unknown_pipelines,
            assigned_pipelines=assigned_pipelines,
            chromosome=chromosome,
            project_data=project_data,
            worker_group=worker_group,
            worker_team_id=earliest_worker.team_id
        )

        if next_pipeline_no is None:
            # 这个焊工找不到能做的管线了，标记它
            workers_with_no_task.add(earliest_worker.worker_id)
            continue

        # 获取管线对象
        pipeline = project_data.get_pipeline(next_pipeline_no)
        if pipeline is None:
            assigned_pipelines.add(next_pipeline_no)
            continue

        # 计算所需工作天数
        # 使用该焊工的 daily_capacity（支持不同队伍有不同效率）
        # 例如：50寸，该焊工每天能焊25寸，需要2天
        worker_capacity = getattr(earliest_worker, 'daily_capacity',
daily_capacity)
        days_needed = pipeline.total_inches / worker_capacity

        # 分配任务
        start_time = earliest_worker.available_time
        end_time = start_time + days_needed

        # 创建任务
        task = Task(
            pipeline_no=pipeline.pipeline_no,
            worker_id=earliest_worker.worker_id,
            team_id=earliest_worker.team_id,
            start_time=start_time,
            end_time=end_time,
            total_inches=pipeline.total_inches,
            package_no=pipeline.package_no,
            unit_name=pipeline.unit_name
        )

        schedule.add_task(task)
```

南京南化建设有限公司 夏斐 | 5299

```
        # 更新状态
        earliest_worker.available_time = end_time
        assigned_pipelines.add(next_pipeline_no)
        worker_current_package[earliest_worker.worker_id] =
pipeline.package_no
        worker_current_unit[earliest_worker.worker_id] =
pipeline.unit_name

        # 成功分配任务后，清除该焊工的"无任务"标记
        # （因为分配了新任务后，状态可能改变，可能又有新的可做任务了）
        if earliest_worker.worker_id in workers_with_no_task:
            workers_with_no_task.remove(earliest_worker.worker_id)

    return schedule


def _select_next_pipeline(
    current_package: str,
    current_unit: str,
    package_pipelines: Dict[str, List[str]],
    unknown_pipelines: List[str],
    assigned_pipelines: Set[str],
    chromosome: List[str],
    project_data,
    worker_group,
    worker_team_id: int
) -> str:
    """
    根据试压包和单元优先规则选择下一条管线（同时考虑队伍-单元限制）

    优先级：
    1. 当前试压包内未分配且该队伍可以焊的管线（如果有）
    2. 当前单元内未分配且该队伍可以焊的管线（试压包焊完后）
    3. 染色体中下一条未分配且该队伍可以焊的管线（非Unknown）
    4. Unknown管线且该队伍可以焊（最后处理）
    """
    # 优先级1: 当前试压包内还有未分配的管线
    if current_package and current_package != 'Unknown':
        for pipeline_no in package_pipelines[current_package]:
            if pipeline_no not in assigned_pipelines:
                pipeline = project_data.get_pipeline(pipeline_no)
                if pipeline and
worker_group.can_team_work_on_unit(worker_team_id, pipeline.unit_name):
                    return pipeline_no
```

```
    # 优先级2: 当前单元内还有未分配的管线（排除Unknown）
    if current_unit:
        for pipeline_no in chromosome:
            if pipeline_no not in assigned_pipelines:
                # 检查是否是Unknown
                if pipeline_no in unknown_pipelines:
                    continue
                # 获取管线对象
                pipeline = project_data.get_pipeline(pipeline_no)
                if pipeline and pipeline.unit_name == current_unit:
                    if
worker_group.can_team_work_on_unit(worker_team_id, pipeline.unit_name):
                        return pipeline_no

    # 优先级3: 按染色体顺序选择下一条未分配的管线（非Unknown）
    for pipeline_no in chromosome:
        if pipeline_no not in assigned_pipelines:
            # 检查是否是Unknown
            is_unknown = pipeline_no in unknown_pipelines
            if not is_unknown:
                pipeline = project_data.get_pipeline(pipeline_no)
                if pipeline and
worker_group.can_team_work_on_unit(worker_team_id, pipeline.unit_name):
                    return pipeline_no

    # 优先级4: Unknown管线
    for pipeline_no in unknown_pipelines:
        if pipeline_no not in assigned_pipelines:
            pipeline = project_data.get_pipeline(pipeline_no)
            if pipeline and
worker_group.can_team_work_on_unit(worker_team_id, pipeline.unit_name):
                return pipeline_no
```

4. 2025.11.10
   a. 共现矩阵生成脚本

复制代码

```
"""
单元-天数共现矩阵热力图生成器

功能：读取调度方案Excel，生成单元-天数热力图
- 横坐标：天数（第1天、第2天...）
```

```
- 纵坐标：单元名称
- 单元格值：该天该单元有多少焊工在工作
- 颜色：焊工数越多，颜色越深

使用方法：
    python 生成共现矩阵热力图.py [调度方案文件路径]

    如果不提供文件路径，默认读取 "output_files/最优调度方案_汇总.xlsx"
"""

import sys
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path
from math import ceil, floor

# 输出文件夹
OUTPUT_DIR = "output_files"


def load_schedule(file_path: str) -> pd.DataFrame:
    """
    加载调度方案Excel文件

    参数:
        file_path: Excel文件路径

    返回:
        DataFrame，包含调度方案数据
    """
    if not Path(file_path).exists():
        raise FileNotFoundError(f"找不到文件: {file_path}")

    df = pd.read_excel(file_path, sheet_name='调度方案')

    # 检查必需的列
    required_columns = ['管线号', '焊工ID', '开始时间', '结束时间', '单元名称']
    missing_columns = [col for col in required_columns if col not in df.columns]

    if missing_columns:
```

```
        raise ValueError(f"Excel文件缺少以下列: {missing_columns}")

    print(f"成功加载调度方案，共 {len(df)} 条任务记录")
    return df


def calculate_unit_day_matrix(df: pd.DataFrame) -> pd.DataFrame:
    """
    计算单元-天数共现矩阵

    逻辑:
    - 对每个任务，计算它覆盖的天数范围（向下取整开始，向上取整结束）
    - 只要任务时间与某天有重叠，就算该焊工在该天该单元工作
    - 同一个焊工在同一天可以被多个单元统计（重复计数）

    参数:
        df: 调度方案DataFrame

    返回:
        单元-天数矩阵DataFrame（行=单元，列=天数）
    """
    # 计算最大天数（向上取整）
    max_day = int(ceil(df['结束时间'].max()))

    # 初始化矩阵：{(unit, day): worker_set}
    unit_day_workers = {}

    for _, task in df.iterrows():
        unit_name = task['单元名称']
        worker_id = task['焊工ID']
        start_time = task['开始时间']
        end_time = task['结束时间']

        # 计算任务覆盖的天数范围
        # 如果任务在 5.3-6.2 天，应该覆盖第6天和第7天
        start_day = int(floor(start_time)) + 1  # 第几天（从1开始）
        end_day = int(ceil(end_time))

        # 对于每一天，记录该焊工在该单元工作
        for day in range(start_day, end_day + 1):
            if day > max_day:
                break
            key = (unit_name, day)
            if key not in unit_day_workers:
                unit_day_workers[key] = set()
```

南京南化建设有限公司 夏斐 | 5299

```
            unit_day_workers[key].add(worker_id)

    # 转换为矩阵
    units = sorted(df['单元名称'].unique())
    days = list(range(1, max_day + 1))

    matrix = []
    for unit in units:
        row = []
        for day in days:
            key = (unit, day)
            worker_count = len(unit_day_workers.get(key, set()))
            row.append(worker_count)
        matrix.append(row)

    # 创建DataFrame
    matrix_df = pd.DataFrame(matrix, index=units, columns=[f'第{d}天'
for d in days])

    print(f"\n共现矩阵统计:")
    print(f"  单元数量: {len(units)}")
    print(f"  天数范围: 1-{max_day} 天")
    print(f"  最大焊工数: {matrix_df.max().max()}")

    return matrix_df


def plot_heatmap(matrix_df: pd.DataFrame, output_path: str = "单元-天数热
力图.png"):
    """
    绘制热力图

    参数:
        matrix_df: 单元-天数矩阵
        output_path: 输出图片路径
    """
    # 设置中文字体
    plt.rcParams['font.sans-serif'] = ['Arial Unicode MS', 'SimHei',
'STSong']
    plt.rcParams['axes.unicode_minus'] = False

    # 计算图片大小（根据数据量自适应）
    n_units = len(matrix_df)
    n_days = len(matrix_df.columns)
```

```
    # 每个单元至少0.4英寸，每天至少0.3英寸
    fig_height = max(8, n_units * 0.4)
    fig_width = max(12, n_days * 0.3)

    # 创建图表
    plt.figure(figsize=(fig_width, fig_height))

    # 绘制热力图
    ax = sns.heatmap(
        matrix_df,
        cmap='YlOrRd',  # 黄-橙-红配色
        annot=True,     # 显示数值
        fmt='g',        # 数值格式
        cbar_kws={'label': '焊工数量'},
        linewidths=0.5,
        linecolor='gray'
    )

    # 设置标题和标签
    plt.title('单元-天数焊工分布热力图', fontsize=16, pad=20)
    plt.xlabel('天数', fontsize=12)
    plt.ylabel('单元名称', fontsize=12)

    # 旋转x轴标签
    plt.xticks(rotation=45, ha='right')
    plt.yticks(rotation=0)

    # 调整布局
    plt.tight_layout()

    # 保存图片
    plt.savefig(output_path, dpi=300, bbox_inches='tight')
    print(f"\n热力图已保存到: {output_path}")

    # 显示图片（可选）
    # plt.show()
    plt.close()


def main():
    """
    主函数
    """
    # 获取文件路径（从命令行参数或使用默认值）
    if len(sys.argv) > 1:
```

南京南化建设有限公司 夏斐 | 5299

```
    file_path = sys.argv[1]
else:
    # 默认从 output_files 文件夹读取
    file_path = os.path.join(OUTPUT_DIR, "最优调度方案_汇总.xlsx")

print("=" * 60)
print("单元-天数共现矩阵热力图生成器")
print("=" * 60)

try:
    # 1. 加载调度方案
    print(f"\n[步骤1] 加载调度方案: {file_path}")
    df = load_schedule(file_path)

    # 2. 计算共现矩阵
    print("\n[步骤2] 计算单元-天数共现矩阵...")
    matrix_df = calculate_unit_day_matrix(df)

    # 3. 绘制热力图（保存到 output_files 文件夹）
    print("\n[步骤3] 生成热力图...")
    output_path = os.path.join(OUTPUT_DIR, "单元-天数热力图.png")
    plot_heatmap(matrix_df, output_path)

    # 4. 导出矩阵数据（保存到 output_files 文件夹）
    matrix_output = os.path.join(OUTPUT_DIR, "单元-天数矩阵.xlsx")
    matrix_df.to_excel(matrix_output)
    print(f"矩阵数据已导出到: {matrix_output}")

    print("\n" + "=" * 60)
    print("处理完成！")
    print("=" * 60)
    print(f"\n生成的文件：")
    print(f"  - {output_path}")
    print(f"  - {matrix_output}")

except Exception as e:
    print(f"\n错误: {str(e)}")
    import traceback
    traceback.print_exc()
    sys.exit(1)
```

南京南化建设有限公司 夏斐 | 5299